

Nosite — Feuille de route produit-ready

Découpage actionnable du projet existant jusqu'au SaaS acceptant ses premiers clients payants. Constats vérifiés par lecture du code au commit réel, milestones séquencées par dépendances.

7

MILESTONES

41

ISSUES

2

BLOQUANTS

Sommaire

La feuille de route reprend l'état réel du dépôt à `c7fb6d1` et le séquence des verrous de sécurité jusqu'à la différenciation. Chaque issue est autonome : elle peut être traitée par n'importe quel contributeur à partir de sa seule description, avec ses références `fichier:ligne`, ses tâches, ses critères d'acceptation et ses dépendances.

M7	Verrous de sécurité avant ouverture	7 issues · Jours
M8	Authentification & gestion du compte (Clerk)	5 issues · Semaines
M9	Paieement & facturation (Stripe)	8 issues · Semaines
M10	Monitoring & observabilité	5 issues · Semaines
M11	Design & UX polish	8 issues · Semaines
M12	Documentation & déploiement	4 issues · Continu
M13	Différenciation, continuité & angles morts (phase 2)	4 issues · Trimestre
§	Résumé exécutif & schéma de dépendances	après le sommaire
§	Analyse Design & UX (partiellement à instruire)	avant M7
§	Checklist post-déploiement	après M13
§	Glossaire d'acronymes	fin

Bloquant

 empêche l'ouverture payante

Majeur

 à corriger avant montée en charge

Mineur

 à traiter, non urgent

Cosmétique

 finition

À instruire

 nécessite une vérification live

Résumé exécutif

L'audit post-remédiation a clôturé les 7 bloquants de juin et fait passer Nosite d'un prototype à une base SaaS crédible. Le code a depuis avancé (commit [c7fb6d1](#), « Nosite v0.1 », landing en place, produit renommé). Cinq verrous, tous revérifiés dans le code, séparent encore le projet d'une ouverture payante. Cette roadmap les traite (M7), puis construit l'identité (M8 · Clerk), la monétisation (M9 · Stripe), l'observabilité (M10), la finition (M11), le socle de déploiement (M12) et la différenciation (M13).

VERROUS REVÉRIFIÉS

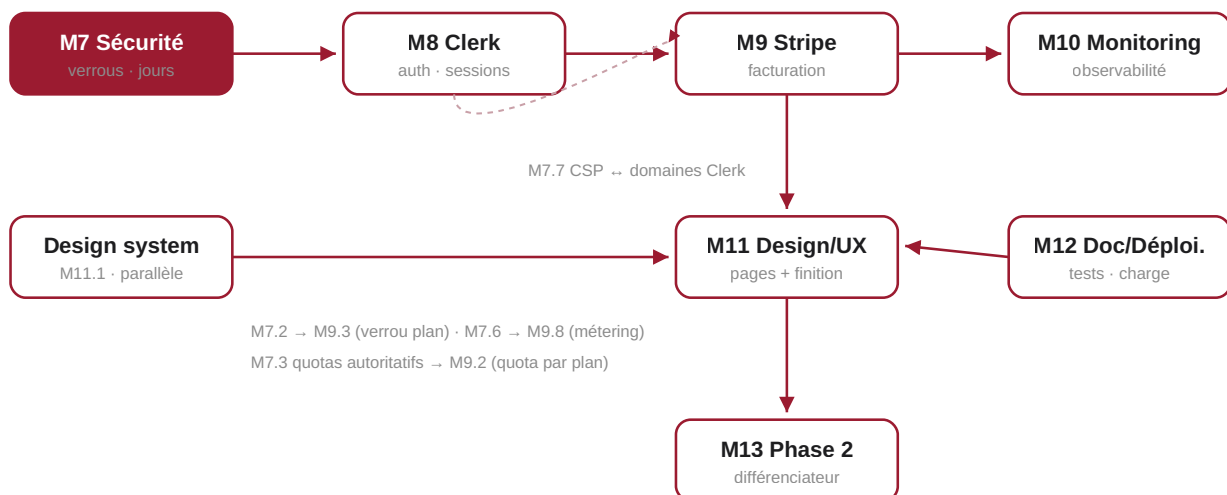
BLOQUANTS MONÉTISATION

ISSUES ACTIONNABLES

DÉCISIONS STRUCTURANTES INTÉGRÉES

Clerk est retenu pour l'authentification : il gère des sessions `httpOnly` et **absorbe** le correctif de vol de JWT, le reset, la force du mot de passe, le changement de mot de passe, le multi-session, le MFA et l'OAuth Google. Le fix JWT manuel quitte donc M7 pour devenir une conséquence de M8. Un seul arbitrage reste ouvert (M8.4) : la protection anti-bot native de Clerk (**Turnstile**) vs **reCAPTCHA v3** demandé. La **section Design & UX** est partiellement « à instruire » : l'audit visuel live n'a pas pu être conduit ; les résidus produit, eux, sont vérifiés dans le code.

SCHÉMA DE DÉPENDANCES



Trait plein = dépendance de séquençement ; pointillés = dépendances croisées d'issues. Le design system (M11.1) démarre en parallèle de M7 ; les pages transactionnelles (checkout, facturation) ne deviennent réelles qu'après Stripe (M9). M13 est optionnel pour l'ouverture payante mais requis pour une vente/levée.

CHEMIN CRITIQUE VERS LA MONÉTISATION

M7 (verrous, jours) → M8.1 Clerk (identité, débloquent tout) → M9.1 Stripe Checkout + M9.3/M9.4 webhooks (revenu réel) → M9.5/M9.6 factures & e-mails. En parallèle dès le départ : M7.3/M7.4 quotas et rate-limit autoritatifs, sans lesquels le contrôle de coût se relâche à l'échelle et la facturation par plan reste fragile.

RISQUES & ANGLES MORTS NON ADRESSABLES PAR LE CODE SEUL

Quotas/alertes budgétaires Google Cloud (hors dépôt) ; validation côté Stripe (comptes, conformité paiement) ; avis juridique sur l'export Google Places et la base légale RGPD + opt-out tiers ; test de charge réel en conditions multi-instance ; et la décision anti-bot (M8.4). Ces points sont signalés dans les issues concernées plutôt que tranchés ici.

Analyse Design & UX

Avertissement de méthode. L'audit visuel fin (rendu live, contraste écran par écran, micro-interactions) n'a pas pu être conduit en navigation réelle authentifiée. Cette section s'appuie sur la lecture du code (`globals.css`, composants, états gérés) et doit être complétée par tes observations. Les résidus produit listés en M11 sont, eux, vérifiés dans le code à `c7fb6d1`.

PALETTE & TOKENS RÉELLEMENT CONSTATÉS

Accent **émeraude** : `#16A34A` (clair) / `#22C55E` (sombre), avec déclinaisons d'opacité (`--accent-d/-s/-border`). Fond chaud `#FAFAF9` / `#F4F2EE`. Échelle de texte Zinc : `--t1 #09090B`, `--t2 #52525B`, `--t3 #71717A`. **Dark mode** présent via `[data-theme="dark"]`, avec remappage de l'échelle « emerald » de Tailwind sur l'accent. Le système de tokens est donc cohérent et déjà en place.

AXES D'AMÉLIORATION IDENTIFIÉS (DEPUIS LE CODE)

Pages manquantes pour un SaaS payant : `/pricing` dédiée (seule une ancre `#tarifs` existe), `/account/billing` (la page `settings` n'a ni changement de mot de passe ni factures). Résidus de parcours vérifiés : liens profonds cassés au-delà du TTL (`results/[id]/page.tsx`), ajout au CRM automatique à l'ouverture de fiche (`DetailClient.tsx:48-50`), et confusion « bloqué anti-bot » / « obsolète » dans le classement. Accessibilité à porter au niveau AA. Couverture complète des états de composants (`disabled/success`) à confirmer.

PROPOSITIONS

Traiter ces points dans M11 (finition) en s'appuyant sur le design system existant, et ajouter les sections de conversion à la landing (avis clients, promesses, plans). Le design system pouvant tourner en parallèle de M7, l'instruction visuelle live peut commencer immédiatement, indépendamment des chantiers backend.

Tout ce qui est indépendant du modèle d'authentification et qui débloque une bêta fermée / facturation pilote. Le correctif JWT `httpOnly` ne figure pas ici : il est absorbé par la migration Clerk (M8), qui pose ses propres cookies de session sécurisés. M7 ferme les fuites, rend les garde-fous de coût autoritatifs et durcit la surface HTTP.

Majeur ×5

Mineur ×2

M7.1 Authentifier ou retirer l'endpoint `/api/usage` global

Majeur

CONSTAT

`backend/src/index.ts:34` expose `app.get('/api/usage', ...)` qui renvoie `{ global: getUsage(), users: getAllUsersUsage() }` **sans** `requireAuth`, alors que `/api/usage/me` (ligne 38) est, lui, protégé. N'importe qui peut donc lister les identifiants de tous les utilisateurs et leur consommation. Fuite de données et cartographie d'usage offerte à un concurrent.

OBJECTIF

L'endpoint global de consommation n'est accessible qu'à un rôle exploitant authentifié, ou il est supprimé.

TÂCHES

- Décider : conserver l'endpoint pour l'ops (alors le protéger) ou le supprimer.
- S'il est conservé : exiger `requireAuth` + un contrôle de rôle administrateur (`claim/role`).
- S'il est supprimé : retirer la route et la fonction `getAllUsersUsage` si plus référencée.
- Vérifier qu'aucune page frontend n'appelle l'endpoint global non protégé.

CRITÈRES D'ACCEPTATION

- ✓ Une requête anonyme sur `/api/usage` renvoie 401/404, jamais la liste des utilisateurs.
- ✓ Un test d'intégration couvre le rejet anonyme.

DÉPENDANCES

Indépendante. Le contrôle de rôle s'aligne ensuite sur Clerk (M8) si un rôle admin Clerk est retenu.

FICHIERS IMPACTÉS : `backend/src/index.ts:34-40` · `backend/src/services/userQuota.ts` (`getAllUsersUsage`)

M7.2 Verrouiller le changement de plan (`PATCH /api/plans/me`) — correctif intérimaire

Majeur

CONSTAT

`backend/src/routes/plans.ts` : `PATCH /me` est authentifié (`requireAuth`) mais valide seulement l'existence du `planId` puis appelle `setUserPlan` — **aucun contrôle de paiement**. Tout utilisateur connecté s'attribue gratuitement « Business ». Les quotas par plan sont donc décoratifs et la monétisation impossible.

OBJECTIF

Tant que Stripe n'est pas branché (M9), un utilisateur ne peut pas s'attribuer un plan payant ; seul un downgrade vers Free (ou aucun changement) est permis.

TÂCHES

- Introduire une notion de « plan payant » (flag sur le plan) côté `planStore`.
- Dans `PATCH /me` : refuser (403) tout passage vers un plan payant sans abonnement actif.
- Autoriser le retour à Free.
- Préparer le point d'extension où M9 injectera la vérification d'abonnement Stripe.

CRITÈRES D'ACCEPTATION

- ✓ Un `PATCH /me` vers « Business » sans abonnement renvoie 403.
- ✓ Le retour vers Free reste possible.
- ✓ Un test couvre le refus d'auto-attribution payante.

DÉPENDANCES

Complétée par M9.3 (verrou définitif piloté par les webhooks Stripe).

FICHIERS IMPACTÉS : `backend/src/routes/plans.ts` · `backend/src/services/planStore.ts`

M7.3 Rendre les quotas Google & par utilisateur autoritatifs (incrément atomique)

Majeur

CONSTAT

`backend/src/services/googleQuota.ts` tient un compteur primaire **en mémoire** (`let counter`, `DEFAULT_DAILY_LIMIT = 500`) avec un simple repli DB ; `userQuota.ts` utilise `memCounters = new Map()` + un `INSERT ... ON CONFLICT` mais lit aussi un état mémoire (`/* fallback to in-memory */`). En multi-instance, chaque instance a son propre compteur → la limite effective devient $\approx N \times \text{limite}$ et le contrôle de coût se relâche dès le second worker.

OBJECTIF

Le quota (Google/jour et par utilisateur/jour) est la source de vérité en base/Redis, incrémenté atomiquement, partagé entre instances.

TÂCHES

- Faire de l'incrément un `UPDATE ... RETURNING` atomique (Postgres) ou un `INCR` Redis avec expiration jour.
- Lire le quota depuis le store partagé avant chaque appel payant, pas depuis la mémoire.
- Conserver un repli mémoire uniquement comme dégradé explicite et journalisé, pas comme source.
- Couvrir le décompte concurrent par un test.

CRITÈRES D'ACCEPTATION

- ✓ Deux instances partagent le même compteur quotidien (vérifié en test d'intégration).
- ✓ Le dépassement bloque l'appel Google quel que soit le worker.
- ✓ Aucune divergence de compteur après redéploiement.

DÉPENDANCES

Requiert Redis et/ou Postgres déjà provisionnés (présents : `ioredis`, `pg`).

FICHIERS IMPACTÉS : `backend/src/services/googleQuota.ts` · `backend/src/services/userQuota.ts`

M7.4 Externaliser le store du rate-limiter (Redis)

Majeur

CONSTAT

`backend/src/middleware/rateLimit.ts` instancie `express-rate-limit` **sans store Redis** → store mémoire par défaut : compteur par instance, remis à zéro à chaque redéploiement. La clé est correcte (`req.user?.sub ?? req.ip`) et `trust proxy` est posé ; il ne manque que la persistance partagée.

OBJECTIF

Le rate-limiting survit aux redéploiements et est partagé entre instances.

TÂCHES

- Brancher un store Redis (`rate-limit-redis`) sur `searchRateLimiter` et `authRateLimiter`.
- Réutiliser le client `ioredis` existant.
- Vérifier le comportement en multi-instance et après restart.

CRITÈRES D'ACCEPTATION

- ✓ Le quota par minute est partagé entre deux instances.
- ✓ Un redéploiement ne réinitialise pas les fenêtres en cours.

DÉPENDANCES

S'appuie sur la même brique Redis que M7.3.

FICHIERS IMPACTÉS : `backend/src/middleware/rateLimit.ts` · `backend/src/index.ts`

M7.5 Partager l'état du circuit-breaker Google Places

Mineur

CONSTAT

`backend/src/services/places.ts:83-84` : `let consecutiveFailures` et `let breakerOpenUntil` sont des variables de module, purement en mémoire. En multi-instance, chaque worker a son propre disjoncteur : une instance peut continuer à marteler Google alors qu'une autre a déjà ouvert son circuit.

OBJECTIF

L'état ouvert/fermé du circuit-breaker est partagé, pour qu'une panne Google soit honorée par toutes les instances.

TÂCHES

- Stocker `breakerOpenUntil` et le compteur d'échecs dans Redis (clé courte, TTL = fenêtre de reset).
- Lire l'état partagé avant chaque appel.
- Conserver le seuil et le délai de reset actuels.

CRITÈRES D'ACCEPTATION

- ✓ Une instance qui ouvre le circuit le propage aux autres.
- ✓ Le comportement fonctionnel (seuil, reset) est préservé.

DÉPENDANCES

S'appuie sur Redis (M7.3/M7.4).

FICHIERS IMPACTÉS : `backend/src/services/places.ts:81-102`

M7.6 Fiabiliser le métering (compteur retourné, pas delta global)

Mineur

CONSTAT

`backend/src/routes/search.ts:120-122` mesure les appels par delta d'un compteur global (`globalBefore = getUsage().total ... globalAfter = getUsage().total`) ; même schéma dans `establishment.ts`. Sous concurrence, le delta capte aussi les appels d'autres requêtes → attribution d'usage imprécise, problématique dès qu'on facture à l'usage.

OBJECTIF

Chaque requête se voit attribuer exactement le nombre d'appels Google qu'elle a déclenchés.

TÂCHES

- Faire remonter le nombre d'appels effectués depuis `nearbySearch/fetchPlaceDetails` (valeur retournée).
- Attribuer ce nombre à l'utilisateur, sans passer par un delta global.
- Couvrir l'attribution sous concurrence par un test.

CRITÈRES D'ACCEPTATION

- ✓ L'attribution d'appels d'une requête est exacte même si d'autres requêtes tournent en parallèle.
- ✓ Un test concurrent valide l'absence de fuite de comptage.

DÉPENDANCES

Préalable à la répercussion de coût (M9.8).

FICHIERS IMPACTÉS : `backend/src/routes/search.ts:120-124` · `backend/src/routes/establishment.ts` · `backend/src/services/places.ts`

M7.7 Durcir la CSP : retrait de `unsafe-inline` / `unsafe-eval`

Majeur

CONSTAT

`frontend/next.config.ts:21` : `script-src 'self' 'unsafe-inline' 'unsafe-eval'` (et `style-src 'unsafe-inline'` ligne 22). Une CSP aussi permissive neutralise la protection XSS : couplée à un token volable, une injection suffit à exfiltrer la session. À traiter pour réduire la surface, indépendamment de l'auth.

OBJECTIF

La CSP n'autorise plus l'exécution de scripts inline/eval ; les scripts légitimes passent par nonce.

TÂCHES

- Passer à une CSP par nonce (Next : `headers()` + nonce propagé aux scripts).
- Retirer `unsafe-eval` et `unsafe-inline` de `script-src`.
- **Coordonner avec Clerk (M8)** : ajouter les domaines de script Clerk requis à la `script-src/` `connect-src/frame-src` (sinon la migration casse l'auth).
- Vérifier que le rendu et les widgets tiers passent sous la nouvelle CSP.

CRITÈRES D'ACCEPTATION

- ✓ Aucune directive `unsafe-inline/unsafe-eval` dans `script-src` en production.
- ✓ Les pages et les composants Clerk se chargent sans violation CSP (vérifié via la console).

DÉPENDANCES

Couplée à M8 : la liste blanche de domaines dépend de la mise en place de Clerk. Faire l'ossature ici, finaliser la liste après M8.1.

FICHIERS IMPACTÉS : `frontend/next.config.ts:14-24`

Décision retenue : **Clerk** remplace l'authentification JWT maison. Clerk gère les sessions via ses propres cookies `httpOnly`, ce qui **résout le vecteur de vol de JWT** (`login/page.tsx:52`) sans correctif manuel. Clerk absorbe aussi nativement le reset de mot de passe, la politique de force, le changement de mot de passe, la gestion multi-sessions, le MFA et l'OAuth Google. Les issues ci-dessous reflètent cette réalité : il s'agit d'intégrer et de configurer Clerk, pas de reconstruire ces briques à la main.

Bloquant ×1

Majeur ×3

Mineur ×1

M8.1 Intégrer Clerk & migrer les sessions (remplace le JWT `document.cookie`)

Bloquant

CONSTAT

Aujourd'hui le token est posé côté navigateur par `frontend/app/login/page.tsx:52` (`document.cookie = pw_token=...; SameSite=Lax`) **sans `httpOnly` ni `Secure`** ; la route `frontend/app/api/auth/login/route.ts` ne fait que proxifier sans poser de cookie serveur. Le token est donc lisible par tout script (XSS = vol de session). Côté backend, `requireAuth.ts` vérifie un JWT maison. Aucune dépendance Clerk n'est installée.

OBJECTIF

Les sessions sont gérées par Clerk (cookies `httpOnly; Secure`), le JWT maison est retiré du chemin d'authentification, et toutes les routes protégées valident la session Clerk.

TÂCHES

- Installer Clerk (frontend `@clerk/nextjs`, vérification de session côté backend).
- Remplacer les écrans `/login` et inscription par les composants Clerk (`<SignIn/>`, `<SignUp/>`).
- Adapter `frontend/middleware.ts` pour la protection de routes Clerk.
- Backend : valider le jeton de session Clerk dans `requireAuth.ts` (à la place du JWT maison).
- Conserver le secret interne `x-internal-secret` en défense en profondeur.
- **Remplacer la détection de connexion**
`document.cookie.includes('pw_token=')` (`page.tsx:12`) par l'état Clerk (`useUser/auth()`), car un cookie `httpOnly` n'est plus lisible par JS.
- Migrer les comptes existants (email/mot de passe) vers Clerk ou prévoir une bascule au prochain login.

CRITÈRES D'ACCEPTATION

- ✓ Aucune session n'est portée par un cookie lisible en JS ; `document.cookie` ne contient plus le token.
- ✓ Une route protégée rejette une session Clerk absente/invalid (401).
- ✓ L'état « connecté » de l'UI ne dépend plus d'un cookie lisible par JS.
- ✓ Un test couvre l'accès protégé avec/sans session valide.

DÉPENDANCES

Débloque M8.2–M8.5. Fournit la liste de domaines pour finaliser M7.7 (CSP).

FICHIERS IMPACTÉS : `frontend/app/login/page.tsx` · `frontend/app/api/auth/*` · `frontend/middleware.ts` · `frontend/app/page.tsx:12` · `backend/src/middleware/requireAuth.ts` · `backend/src/routes/auth.ts`

M8.2 Activer la connexion Google (OAuth) via Clerk

Majeur

CONSTAT

L'auth actuelle est e-mail/mot de passe uniquement (`backend/src/routes/auth.ts`). Aucun fournisseur social. « Sign in with Google » est une attente explicite et un standard d'onboarding B2B.

OBJECTIF

Un utilisateur peut créer un compte et se connecter via Google en un clic ; les comptes e-mail et Google d'une même adresse sont fusionnés.

TÂCHES

- Activer le connecteur Google OAuth dans le dashboard Clerk (client ID/secret Google).
- Ajouter le bouton Google aux écrans Clerk.
- Configurer la fusion de comptes (même e-mail) dans Clerk.
- Vérifier le mapping vers le modèle `users` applicatif (création à la première connexion).

CRITÈRES D'ACCEPTATION

- ✓ La connexion Google crée/retrouve le bon utilisateur applicatif.
- ✓ Un compte e-mail et un compte Google de même adresse ne se dupliquent pas.

DÉPENDANCES

Dépend de M8.1.

FICHIERS IMPACTÉS : Dashboard Clerk · frontend (écrans Clerk) · `backend/src/services/userStore.ts`

M8.3 Politique de mot de passe, force & réinitialisation (Clerk + Resend)

Majeur

CONSTAT

La validation actuelle est faible : `backend/src/routes/auth.ts:26` exige seulement `password.length < 6` → 400, sans regex majuscule/minuscule/caractère spécial ; le frontend pose `minLength={6}` mais affiche « 8 caractères minimum » (`login/page.tsx`) — incohérent. **Aucune** route `reset/forgot` n'existe. Clerk fournit nativement la politique de complexité, l'indicateur de force et le flux de réinitialisation.

OBJECTIF

La complexité (≥ 8 , majuscule, minuscule, caractère spécial) est imposée par Clerk avec indicateur de force en temps réel, et la réinitialisation par e-mail fonctionne (expiration courte).

TÂCHES

- Configurer la politique de mot de passe dans Clerk (longueur min 8, complexité).
- Utiliser le composant Clerk qui affiche la force en temps réel.
- Activer le flux « mot de passe oublié » de Clerk (lien à expiration).
- Brancher Resend comme fournisseur d'e-mail (SMTP/domaine) pour les e-mails Clerk, ou aligner le branding.
- Retirer la validation maison divergente de `auth.ts` une fois Clerk en place.

CRITÈRES D'ACCEPTATION

- ✓ Un mot de passe non conforme est refusé à l'inscription avec un message clair.
- ✓ La force du mot de passe s'affiche pendant la saisie.
- ✓ Un lien de réinitialisation expiré est refusé ; un lien valide permet le changement.

DÉPENDANCES

Dépend de M8.1. E-mails liés à Resend (cf. M9.6).

FICHIERS IMPACTÉS : Dashboard Clerk · frontend (écrans Clerk) · `backend/src/routes/auth.ts` (retrait validation maison)

M8.4 Protection anti-bot — point de décision : Turnstile (Clerk) vs reCAPTCHA v3

Majeur

CONSTAT

Aucune protection anti-bot n'existe sur l'inscription/connexion (aucune dépendance reCAPTCHA). Clerk intègre nativement une protection anti-bot via **Cloudflare Turnstile**. La demande initiale visait **reCAPTCHA v3** (silencieux, score) — qui n'est pas le mécanisme natif de Clerk. Imposer reCAPTCHA v3 implique des flux custom qui contournent les écrans Clerk.

OBJECTIF

Les points d'entrée (inscription, connexion, réinitialisation) sont protégés contre l'automatisation, sans friction visible.

TÂCHES

- **Décider** : (a) activer la protection anti-bot native de Clerk (Turnstile) — chemin de moindre effort, cohérent avec M8 ; ou (b) conserver reCAPTCHA v3 et accepter des flux d'auth partiellement custom.
- Implémenter l'option retenue sur inscription/connexion/forgot.
- Vérifier le score/seuil et le comportement sur faux positifs.

CRITÈRES D'ACCEPTATION

- ✓ Les trois points d'entrée rejettent un volume automatisé en test.
- ✓ Aucun défi visible pour un utilisateur légitime (score-based/silencieux).

DÉPENDANCES

Dépend de M8.1. **Arbitrage à trancher par le porteur produit.**

FICHIERS IMPACTÉS : Dashboard Clerk · frontend (écrans d'auth)

M8.5 Page compte via Clerk <UserProfile/> (mot de passe, sessions, MFA)

Mineur

CONSTAT

`frontend/app/settings/page.tsx` affiche le plan et le prix mais **n'offre ni changement de mot de passe, ni gestion des sessions, ni MFA**. Clerk fournit `<UserProfile/>` qui couvre nativement ces trois besoins.

OBJECTIF

L'utilisateur gère son compte (mot de passe, sessions actives, MFA optionnel) depuis une page dédiée.

TÂCHES

- Intégrer `<UserProfile/>` de Clerk dans la page compte.
- Activer le MFA optionnel (TOTP) dans Clerk.
- Conserver/relire l'affichage du plan (lien avec M9.5).
- Harmoniser le style avec le design system (M11).

CRITÈRES D'ACCEPTATION

- ✓ Un utilisateur peut changer son mot de passe (après vérification de l'actuel).
- ✓ Un utilisateur peut voir/révoquer ses sessions et activer le MFA.

DÉPENDANCES

Dépend de M8.1 ; cohabite avec la page facturation M9.5.

FICHIERS IMPACTÉS : `frontend/app/settings/page.tsx`

Transformer les plans décoratifs en revenu réel. Aucune dépendance Stripe n'est installée ; les plans existent (`planStore`) mais sans paiement. Cette milestone branche Stripe, lie les plans aux quotas Google réels, verrouille définitivement le changement de plan derrière l'abonnement, et expose factures et emails transactionnels.

Bloquant ×1**Majeur** ×5**Mineur** ×2

M9.1 Intégrer Stripe Checkout / Billing Portal

Bloquant

CONSTAT

Aucune intégration de paiement (`stripe` absent des dépendances). Le changement de plan est gratuit (`plans.ts`, cf. M7.2). Sans paiement, aucune monétisation possible.

OBJECTIF

Un utilisateur souscrit un plan payant via Stripe Checkout et gère son abonnement via le Billing Portal.

TÂCHES

- Créer les produits/prix Stripe (Pro, Business) — mensuel.
- Endpoint backend créant une session Stripe Checkout pour le plan choisi.
- Rediriger vers Checkout puis retour succès/annulation.
- Exposer le Billing Portal Stripe pour la gestion (changement, annulation, moyen de paiement).
- Stocker l'`customer/subscription` Stripe par utilisateur.

CRITÈRES D'ACCEPTATION

- ✓ Un paiement test crée un abonnement actif et reflète le plan côté applicatif.
- ✓ L'annulation via le portail repasse l'utilisateur en Free à l'échéance.

DÉPENDANCES

Préalable à M9.2–M9.6. S'appuie sur Clerk (M8) pour l'identité.

FICHIERS IMPACTÉS : `backend/src/routes/plans.ts` (+ nouveau routeur billing) · `backend/src/services/planStore.ts` · `frontend` (page paiement)

M9.2 Lier les plans (Free/Pro/Business) aux quotas Google réels

Majeur

CONSTAT

Les plans portent des libellés/prix (`planStore`, affichés dans `settings`) mais le lien plan → quota Google/utilisateur n'est pas piloté par le plan payé. Le quota par utilisateur est géré dans `userQuota.ts` (avec overrides) indépendamment du plan effectif.

OBJECTIF

Le quota quotidien appliqué à un utilisateur découle de son plan actif (et donc de son paiement).

TÂCHES

- Définir le quota Google/jour par plan (Free/Pro/Business).
- À l'activation/annulation d'abonnement, mettre à jour la limite effective de l'utilisateur (cf. M7.3 autoritatif).
- Refléter le quota restant dans l'UI compte/dashboard.

CRITÈRES D'ACCEPTATION

- ✓ Le quota appliqué correspond au plan payé, vérifié de bout en bout.
- ✓ Un downgrade ajuste le quota au cycle suivant.

DÉPENDANCES

Dépend de M7.3 (quota autoritatif) et M9.1.

FICHIERS IMPACTÉS : `backend/src/services/planStore.ts` · `backend/src/services/userQuota.ts`

M9.3 Verrou définitif du changement de plan derrière l'abonnement (webhooks)

Majeur

CONSTAT

Le correctif intérimaire M7.2 refuse l'auto-attribution payante mais ne connaît pas l'état réel de paiement. C'est Stripe qui détient la vérité (abonnement actif/échu).

OBJECTIF

Le plan effectif d'un utilisateur est strictement déterminé par l'état de son abonnement Stripe.

TÂCHES

- Sur webhook `customer.subscription.created/updated/deleted`, mettre à jour le plan applicatif.
- Faire de `setUserPlan` une opération interne, non déclenchable par l'utilisateur pour un plan payant.
- Gérer les états : actif, en période d'essai, impayé, annulé.

CRITÈRES D'ACCEPTATION

- ✓ Un utilisateur ne peut activer « Business » que via un paiement Stripe réussi.
- ✓ Un abonnement échu/annulé repasse automatiquement le compte en Free.

DÉPENDANCES

Complète M7.2 ; dépend de M9.1 et M9.4.

FICHIERS IMPACTÉS : `backend/src/routes/plans.ts` · (nouveau) `backend/src/routes/stripe-webhook.ts` · `planStore.ts`

M9.4 Webhooks Stripe (cycle de vie de l'abonnement)

Majeur

CONSTAT

Aucun récepteur de webhook n'existe. Sans webhooks, l'application ne sait pas qu'un paiement a réussi, échoué, ou qu'un abonnement a été annulé hors session.

OBJECTIF

L'application réagit de façon fiable aux événements Stripe, avec vérification de signature et idempotence.

TÂCHES

- Endpoint webhook avec vérification de signature Stripe.
- Traiter paiement réussi/échoué, création/maj/suppression d'abonnement, facture payée.
- Garantir l'idempotence (déduplication par event id).
- Journaliser et alerter sur les paiements échoués (lien M10.4).

CRITÈRES D'ACCEPTATION

- ✓ Un événement rejoué n'est pas traité deux fois.
- ✓ Une signature invalide est rejetée.
- ✓ Un paiement échoué déclenche l'état et l'alerte attendus.

DÉPENDANCES

Dépend de M9.1 ; alimente M9.3, M9.5, M9.6.

FICHIERS IMPACTÉS : backend (nouveau routeur webhook) · backend/src/services/planStore.ts

M9.5 Page /account/billing — historique de facturation

Majeur

CONSTAT

Aucune page de facturation. `settings/page.tsx` montre le plan et le prix, sans factures ni moyen de paiement. Un client payant attend de retrouver ses factures.

OBJECTIF

Le client consulte son plan, son moyen de paiement et l'historique de ses factures.

TÂCHES

- Page `/account/billing` listant les factures (via API Stripe / portail).
- Lien vers le Billing Portal pour les actions.
- Afficher l'état de l'abonnement et la prochaine échéance.
- Style aligné sur le design system (M11).

CRITÈRES D'ACCEPTATION

- ✓ Les factures payées apparaissent et sont téléchargeables (via Stripe).
- ✓ L'état d'abonnement affiché correspond à Stripe.

DÉPENDANCES

Dépend de M9.1/M9.4 ; cohabite avec la page compte M8.5.

FICHIERS IMPACTÉS : frontend/app/account/billing/page.tsx (nouveau)

M9.6 Emails transactionnels Resend (bienvenue, paiement, facture, expiration, quota)

Majeur

CONSTAT

Aucun e-mail transactionnel (Resend absent des dépendances). Les moments clés (inscription, paiement, facture, expiration de plan, quota atteint) ne génèrent aucune notification.

OBJECTIF

Les événements clés déclenchent un e-mail Resend approprié, avec des templates cohérents.

TÂCHES

- Intégrer le SDK Resend et un domaine d'envoi vérifié.
- Templates : bienvenue, confirmation de paiement, facture, expiration de plan, quota atteint.
- Déclencher depuis les webhooks Stripe (M9.4) et le métering de quota (M7.3).
- Gérer les échecs d'envoi (retry/log).

CRITÈRES D'ACCEPTATION

- ✓ Chaque événement clé envoie l'e-mail attendu en test.
- ✓ Un échec d'envoi est journalisé et ne bloque pas le flux principal.

DÉPENDANCES

Dépend de M9.4 (paiements) et M7.3 (quota) ; partage Resend avec M8.3.

FICHIERS IMPACTÉS : backend (service email Resend) · templates

M9.7 Page `/pricing` dédiée

Mineur

CONSTAT

La landing (`page.tsx`) propose une ancre `#tarifs` mais **aucune page `/pricing` dédiée**. Une page tarifs claire est attendue pour la conversion et le SEO.

OBJECTIF

Une page `/pricing` présente les plans, leurs quotas et leurs prix, avec CTA vers Checkout.

TÂCHES

- Page `/pricing` construite sur le design system (M11), alimentée par `planStore`.
- CTA déclenchant Stripe Checkout (M9.1) pour les plans payants.
- Cohérence avec la section `#tarifs` de la landing.

CRITÈRES D'ACCEPTATION

- ✓ La page liste les plans réels (source `planStore`).
- ✓ Le CTA lance le bon parcours d'achat.

DÉPENDANCES

Page statique réalisable avant Stripe ; CTA réel dépend de M9.1. S'appuie sur le design system (M11).

FICHIERS IMPACTÉS : frontend/app/pricing/page.tsx (nouveau) · frontend/app/page.tsx (`#tarifs`)

M9.8 Métering fiabilisé pour la répercussion de coût

Mineur

CONSTAT

Le suivi d'usage par delta global (cf. M7.6) ne permet pas une répercussion fine du coût Google par utilisateur/plan. Sans attribution exacte, la tarification à l'usage est fragile.

OBJECTIF

L'usage par utilisateur est exact et exploitable pour la facturation à l'usage ou la gestion de dépassement.

TÂCHES

- S'appuyer sur l'attribution exacte de M7.6.
- Agréger l'usage par utilisateur/plan/période en base.
- Exposer la consommation à l'utilisateur (dashboard) et à l'exploitant.

CRITÈRES D'ACCEPTATION

- ✓ L'usage facturable par utilisateur est reconstituable et exact.
- ✓ Le dépassement est détecté et traité selon le plan.

DÉPENDANCES

Dépend de M7.6 et M9.2.

FICHIERS IMPACTÉS : backend/src/services/userQuota.ts · backend/src/services/db.ts

Rendre l'exploitation visible. Aujourd'hui : aucun outil (Sentry, Mixpanel, Vercel Analytics absents) et l'« alerting » se réduit à `console.error/console.warn` (circuit-breaker, seuil 80 % de quota). Une panne Google ou un pic de coût n'est visible qu'a posteriori dans les logs.

Majeur ×3

Mineur ×2

M10.1 Error tracking Sentry (backend + frontend)

Majeur

CONSTAT

Aucun suivi d'erreurs en production. Les erreurs serveur (ex. `GoogleApiError`, échecs DB) et les erreurs client ne sont ni agrégées ni alertées.

OBJECTIF

Toute erreur non gérée (back et front) est capturée, contextualisée et notifiée.

TÂCHES

- Intégrer Sentry côté backend Express et frontend Next.
- Capturer les erreurs serveur avec contexte (route, utilisateur anonymisé).
- Configurer les alertes (seuils, nouveaux types d'erreurs).
- Vérifier qu'aucune donnée personnelle sensible n'est envoyée.

CRITÈRES D'ACCEPTATION

- ✓ Une erreur provoquée apparaît dans Sentry avec sa stack et son contexte.
- ✓ Une alerte se déclenche sur un seuil défini.

DÉPENDANCES

Indépendante ; complète M10.4.

FICHIERS IMPACTÉS : backend/src/index.ts · frontend (instrumentation Next)

M10.2 Analytics produit Mixpanel (signup, search, upgrade, export)

Majeur

CONSTAT

Aucun tracking d'événements produit. Impossible de mesurer l'activation, la conversion ou l'usage des fonctions clés.

OBJECTIF

Les événements clés du funnel sont suivis et exploitables.

TÂCHES

- Intégrer Mixpanel (frontend, + serveur si pertinent).
- Tracker : inscription, connexion, recherche, ouverture de fiche, ajout CRM, export CSV, upgrade de plan.
- Définir des propriétés cohérentes (plan, source).
- Respecter le consentement (cf. RGPD M13.4).

CRITÈRES D'ACCEPTATION

- ✓ Les événements clés remontent avec leurs propriétés.
- ✓ Le funnel signup → search → upgrade est reconstituable.

DÉPENDANCES

Indépendante ; consentement lié à M13.4.

FICHIERS IMPACTÉS : frontend (instrumentation) · points d'événement (search, export, upgrade)

M10.3 Vercel Analytics (Core Web Vitals)

Mineur

CONSTAT

@vercel/analytics absent. Aucune mesure de performance perçue (LCP, CLS, INP) ni de performance par route.

OBJECTIF

Les Core Web Vitals et la performance par route sont mesurés en continu.

TÂCHES

- Activer Vercel Analytics / Speed Insights sur le frontend.
- Vérifier la collecte des Web Vitals.
- Suivre les routes lentes (ex. résultats).

CRITÈRES D'ACCEPTATION

- ✓ Les Web Vitals sont visibles dans Vercel.
- ✓ Les routes critiques sont suivies.

DÉPENDANCES

Indépendante.

FICHIERS IMPACTÉS : frontend/app/layout.tsx

M10.4 Canal d'alerte actif (e-mail/webhook) — erreurs Google & seuil de quota

Majeur

CONSTAT

L'alerting exploitant se limite à `console.error/console.warn` (ex. circuit-breaker dans `places.ts`, seuil 80 % de quota). Aucun canal actif : une panne Google n'est jamais notifiée, seulement lisible dans les logs.

OBJECTIF

Les événements d'exploitation critiques notifient activement (e-mail/webhook), pas seulement les logs.

TÂCHES

- Brancher un canal d'alerte (e-mail via Resend, ou webhook Slack).
- Déclencher sur : circuit-breaker ouvert, erreurs Google répétées, seuil de quota (80 %/100 %), paiements échoués (M9.4).
- Éviter le bruit (déduplication/seuils).

CRITÈRES D'ACCEPTATION

- ✓ Une panne Google simulée déclenche une alerte active.
- ✓ Le franchissement du seuil de quota notifie l'exploitant.

DÉPENDANCES

S'appuie sur Resend (M9.6) ; complète M10.1.

FICHIERS IMPACTÉS : `backend/src/services/places.ts` · `backend/src/services/googleQuota.ts`

M10.5 Dashboard ops interne (santé fournisseurs, usage par plan)

Mineur

CONSTAT

Aucun tableau de bord d'exploitation. La consommation Google, la santé des fournisseurs et l'usage par plan ne sont pas consultables en un endroit.

OBJECTIF

Un écran interne donne l'état des fournisseurs, la consommation et l'usage par plan.

TÂCHES

- Page interne (protégée par rôle, cf. M7.1) agrégeant : état circuit-breaker, quota Google du jour, usage par plan, erreurs récentes.
- Réutiliser les compteurs autoritatifs (M7.3).
- Restreindre l'accès aux exploitants.

CRITÈRES D'ACCEPTATION

- ✓ L'écran reflète l'état réel (quota, circuit-breaker, usage).
- ✓ L'accès est limité au rôle exploitant.

DÉPENDANCES

Dépend de M7.1 (rôle) et M7.3 (compteurs).

FICHIERS IMPACTÉS : `frontend (page ops)` · `backend (endpoints agrégés)`

Section partiellement à instruire. L'audit visuel fin (rendu live, micro-interactions, contraste écran par écran) n'a pas pu être conduit en navigation réelle ; il s'appuie ici sur la lecture du code (Tailwind, tokens, états gérés) et devra être complété par tes propres observations. Les résidus produit ci-dessous, eux, sont vérifiés dans le code à `c7fb6d1`. Palette réelle constatée : accent émeraude `#16A34A` (clair) / `#22C55E` (sombre), fond `#FAFAF9`, échelle de texte Zinc (`--t1 #09090B`, `--t2`, `--t3`), dark mode via `[data-theme]`.

Majeur ×2

Mineur ×4

Cosmétique ×1

À instruire ×1

M11.1 Audit visuel de cohérence (palette, typographie, espacements)

À instruire

CONSTAT

La palette et les tokens existent (`globals.css` : accent émeraude, échelle Zinc, dark mode). La cohérence d'application écran par écran (hiérarchie, contraste, rythme vertical) **n'a pas été vérifiée en live** et reste à instruire.

OBJECTIF

Un langage visuel cohérent et documenté, appliqué uniformément sur toutes les pages.

TÂCHES

- Recenser les composants et vérifier l'usage homogène des tokens.
- Documenter la hiérarchie typographique et les espacements.
- Lister les écarts (couleurs hors palette, tailles ad hoc) à corriger.
- À instruire avec captures réelles des écrans.

CRITÈRES D'ACCEPTATION

- ✓ Un inventaire des écarts visuels est produit.
- ✓ Les tokens sont appliqués sans valeurs ad hoc résiduelles.

DÉPENDANCES

Précède/structure le reste de M11.

FICHIERS IMPACTÉS : `frontend/app/globals.css` · composants UI

M11.2 États de composants complets (disabled, loading, error, success)

Mineur

CONSTAT

Les états de chargement et d'erreur ont progressé (skeleton résultats, bannières), mais la couverture complète des états (disabled, success, hover/focus) sur l'ensemble des formulaires et boutons reste à instruire/compléter.

OBJECTIF

Chaque composant interactif possède ses états visuels complets et cohérents.

TÂCHES

- Définir les états standard pour boutons, champs, cartes.
- Compléter les états manquants (disabled/success notamment).
- Vérifier le feedback (toasts/skeletons) sur les actions clés.

CRITÈRES D'ACCEPTATION

- ✓ Les composants clés exposent tous leurs états.
- ✓ Les actions critiques donnent un feedback visible.

DÉPENDANCES

S'appuie sur M11.1.

FICHIERS IMPACTÉS : composants UI (formulaires, boutons)

M11.3 Accessibilité WCAG 2.1 AA

Majeur

CONSTAT

L'audit post-remédiation note des progrès (aria-hidden sur icônes, badges texte en plus de la couleur) mais l'accessibilité globale (contraste AA, ordre de focus, labels, navigation clavier, lecteur d'écran) n'a pas été vérifiée en live et reste à porter au niveau AA.

OBJECTIF

Les pages clés respectent WCAG 2.1 AA (contraste, focus, labels, alternatives).

TÂCHES

- Vérifier les contrastes (texte/fond, états) au seuil AA.
- Garantir l'ordre de focus et la navigation clavier complète.
- Labels/aria sur tous les contrôles ; alternatives aux signaux couleur seuls.
- Tester au lecteur d'écran sur les parcours principaux.

CRITÈRES D'ACCEPTATION

- ✓ Les parcours clés passent un audit a11y automatisé sans erreur bloquante.
- ✓ La navigation clavier et lecteur d'écran est fonctionnelle sur recherche → résultats → fiche.

DÉPENDANCES

S'appuie sur M11.1.

FICHIERS IMPACTÉS : composants UI · pages search/results/[id]

M11.4 Liens profonds de fiche durables (re-résolution par Place ID)

Majeur

CONSTAT

Vérifié à c7fb6d1 : frontend/app/results/[id]/page.tsx:20 lit sessionStorage('pw_search_results') et affiche « Établissement introuvable ou lien expiré » (ligne 93) ; côté backend, establishment.ts GET /:id ne renvoie que depuis le cache, sinon 404 « Relancez une recherche ». Un lien partagé ou rouvert après expiration du cache est donc cassé.

OBJECTIF

Une fiche se charge à partir de son seul identifiant, indépendamment de la session et du cache.

TÂCHES

- Au cache-miss, re-résoudre la fiche par Place ID via `fetchPlaceDetails` (en respectant quota et conformité).
- Gérer proprement un Place ID réellement introuvable.
- Garantir le partage d'un lien de prospect entre sessions/appareils.

CRITÈRES D'ACCEPTATION

- ✓ Un lien ouvert dans un nouvel onglet/après expiration affiche la fiche.
- ✓ Un Place ID invalide renvoie un état explicite, pas une page cassée.

DÉPENDANCES

Résidu de PW-19. Respecte la conformité Google Places (M13.4).

FICHIERS IMPACTÉS : frontend/app/results/[id]/page.tsx · backend/src/routes/establishment.ts

M11.5 Rendre l'ajout au CRM explicite (vs automatique à l'ouverture)

Mineur

CONSTAT

Vérifié : frontend/app/results/[id]/components/DetailClient.tsx:48-50 ajoute automatiquement le prospect au CRM à l'ouverture de la fiche (`useEffect ... if (!isAdded(e.id)) add(e)`). « Consulter = enregistrer » surprend l'utilisateur et gonfle artificiellement « Mes prospects ».

OBJECTIF

L'ajout au CRM résulte d'une action explicite de l'utilisateur.

TÂCHES

- Remplacer l'ajout automatique par un bouton « Ajouter à mes prospects ».
- Conserver la persistance des notes/statut une fois ajouté.
- Indiquer clairement l'état (ajouté / non ajouté).

CRITÈRES D'ACCEPTATION

- ✓ Ouvrir une fiche n'ajoute pas le prospect sans clic.
- ✓ L'action d'ajout est claire et réversible.

DÉPENDANCES

Indépendante.

FICHIERS IMPACTÉS : frontend/app/results/[id]/components/DetailClient.tsx:48-50

M11.6 Distinguer « bloqué anti-bot » d'« obsolète »

Mineur

CONSTAT

Résidu de PW-16 : un site répondant 403/404 à un robot (anti-scraping) reste « reachable » et finit classé « obsolète » faute d'indices. Le signal de qualification peut donc qualifier à tort un site protégé comme obsolète.

OBJECTIF

Un site qui bloque les robots est étiqueté comme tel, distinct d'« obsolète ».

TÂCHES

- Détecter les réponses anti-bot (403/429/captcha) et créer un état dédié.
- Exclure ces cas du classement « obsolète ».
- Refléter l'état dans l'UI (niveau de confiance).

CRITÈRES D'ACCEPTATION

- ✓ Un site renvoyant 403 à un bot n'est plus classé « obsolète ».
- ✓ L'UI distingue « bloqué » d'« obsolète ».

DÉPENDANCES

Résidu de PW-16 ; touche `websiteChecker`.

FICHIERS IMPACTÉS : `backend/src/services/websiteChecker.ts` · `backend/src/routes/search.ts`

M11.7 Sections landing : avis clients, promesses, plans

Mineur

CONSTAT

La landing (`page.tsx`) possède hero + ancres Produit/Tarifs/FAQ mais les sections « avis clients », « promesses » et « plans » demandées ne sont pas présentes/structurées. Ces sections renforcent la conversion.

OBJECTIF

La landing présente avis clients, promesses produit et aperçu des plans, sur le design system.

TÂCHES

- Concevoir les sections (avis, promesses, plans) avec le design system (M11.1).
- Relier l'aperçu plans à `/pricing` (M9.7).
- Garantir la cohérence responsive.

CRITÈRES D'ACCEPTATION

- ✓ Les trois sections sont présentes et cohérentes visuellement.
- ✓ L'aperçu plans renvoie vers `/pricing`.

DÉPENDANCES

S'appuie sur M11.1 ; relié à M9.7.

FICHIERS IMPACTÉS : `frontend/app/page.tsx`

M11.8 Dark mode & micro-interactions

Cosmétique

CONSTAT

Le dark mode existe (`[[data-theme]]`) ; les micro-interactions (transitions, hover, skeletons) sont partielles.
Finition à instruire en live.

OBJECTIF

Dark mode complet et micro-interactions cohérentes sur les parcours clés.

TÂCHES

- Vérifier le dark mode sur toutes les pages (contraste, états).
- Harmoniser transitions/hover/focus.
- Skeletons cohérents sur les chargements.

CRITÈRES D'ACCEPTATION

- ✓ Le dark mode est complet et lisible partout.
- ✓ Les transitions sont cohérentes et non distrayantes.

DÉPENDANCES

Finition, après M11.1–M11.3.

FICHIERS IMPACTÉS : frontend/app/globals.css · composants

Rendre le déploiement reproductible et l'exploitation tenable. L'audit a montré qu'une doc d'installation fausse casse la prod ; avec Clerk/Stripe/Resend/monitoring, le nombre de secrets explose et doit être documenté et testé.

Majeur ×3

Mineur ×1

M12.1 `.env.example` complet (Clerk, Stripe, Resend, anti-bot, Sentry, Mixpanel, Redis, DB)

Majeur

CONSTAT

L'audit post-remédiation note un `.env.example` déjà complété (clé Google, secrets, JWT, DB, Redis). Mais les nouvelles briques (Clerk, Stripe, Resend, Turnstile/reCAPTCHA, Sentry, Mixpanel) introduisent des secrets non documentés. Suivre une doc incomplète recasse l'installation.

OBJECTIF

Un développeur installe et démarre le projet en suivant la doc à la lettre, toutes briques incluses.

TÂCHES

- Ajouter au `.env.example` (back et front) les variables Clerk, Stripe (clés + webhook secret), Resend, anti-bot, Sentry, Mixpanel.
- Documenter chaque variable (rôle, où la configurer : Vercel vs Railway).
- Vérifier qu'une installation neuve démarre sans 503/erreur de secret.

CRITÈRES D'ACCEPTATION

- ✓ Une installation neuve suivant la doc démarre sans erreur de configuration.
- ✓ Toutes les variables requises par le code figurent dans `.env.example`.

DÉPENDANCES

S'enrichit au fil de M8/M9/M10.

FICHIERS IMPACTÉS : backend/.env.example · frontend/.env.example · README.md

M12.2 Checklist de déploiement (secrets, quotas GCP, alertes, Stripe/Clerk)

Majeur

CONSTAT

Angle mort persistant : les quotas QPD et alertes budgétaires Google Cloud, les restrictions de clé, et la configuration Stripe/Clerk sont hors dépôt et non garantis. Le code pose une barrière applicative, pas la barrière plateforme.

OBJECTIF

Une checklist garantit que chaque déploiement configure les protections plateforme et les secrets.

TÂCHES

- Rédiger la checklist : secrets (Vercel/Railway), quotas QPD GCP + alertes budget, restriction de clé Google, webhooks Stripe, domaines/redirections Clerk, domaine d'envoi Resend.
- Inclure la vérification des protections de coût avant ouverture.
- Versionner la checklist dans le dépôt.

CRITÈRES D'ACCEPTATION

- ✓ La checklist couvre secrets, quotas GCP, alertes, Stripe, Clerk, Resend.
- ✓ Un déploiement suit et coche la checklist.

DÉPENDANCES

Transverse à M8–M10.

FICHIERS IMPACTÉS : docs/déploiement.md (nouveau)

M12.3 Runbook incidents (circuit-breaker, quota Google, paiement échoué)

Mineur

CONSTAT

Aucun runbook documenté pour les incidents courants. Sans procédure, un circuit-breaker ouvert ou un quota Google atteint reste géré au cas par cas.

OBJECTIF

Les incidents fréquents disposent d'une procédure claire de diagnostic et de résolution.

TÂCHES

- Documenter : circuit-breaker ouvert, quota Google atteint, paiement échoué, Redis/DB indisponible.
- Pour chacun : symptômes, diagnostic, actions, escalade.
- Lier aux alertes (M10.4).

CRITÈRES D'ACCEPTATION

- ✓ Chaque incident courant a une procédure actionnable.
- ✓ Le runbook référence les alertes correspondantes.

DÉPENDANCES

S'appuie sur M10.4.

FICHIERS IMPACTÉS : docs/runbook.md (nouveau)

M12.4 Tests étendus (auth Clerk, formulaires/a11y, webhook Stripe, charge)

Majeur

CONSTAT

La base de tests est solide (120 tests à `c94cef9`) mais ne couvre pas les nouvelles briques : intégration Clerk, validation/états/a11y des formulaires, flux de paiement et webhooks Stripe, ni un test de charge multi-utilisateur (quota + rate-limit sous charge).

OBJECTIF

Les briques critiques (auth, paiement, quotas sous charge) sont couvertes par des tests automatisés en CI.

TÂCHES

- Tests d'intégration auth Clerk (accès protégé, session valide/invalidé).
- Tests des formulaires (validation, états, a11y de base).
- Tests du flux de paiement et du webhook Stripe (signature, idempotence).
- Test de charge simulé : quota et rate-limit autoritatifs sous concurrence multi-instance.
- Brancher en CI.

CRITÈRES D'ACCEPTATION

- ✓ La CI exécute les nouveaux tests et échoue en cas de régression.
- ✓ Le test de charge valide le partage des quotas/rate-limit entre instances.

DÉPENDANCES

Dépend de M7.3/M7.4 (autoritatif), M8 (Clerk), M9 (Stripe).

FICHIERS IMPACTÉS : `.github/workflows/ci.yml` · `backend/src/__tests__` / · `frontend (tests)`

Au-delà de la viabilité payante : construire la barrière à l'entrée et couvrir les risques non adressables par du code seul. Ces items ne bloquent pas l'ouverture de la bêta payante mais conditionnent une vente/levée et la robustesse à long terme.

Majeur ×2**Mineur** ×2

M13.1 Enrichissement propriétaire (e-mail, dirigeant, SIRET) — le différenciateur

Majeur

CONSTAT

Résidu de PW-30 : historique, déduplication et relances sont livrés, mais l'enrichissement (e-mail, dirigeant, SIRET) est absent — les actions e-mail ont d'ailleurs été retirées faute de donnée. Sans donnée propriétaire, le produit reste copiable en quelques jours (concurrent prospectweb.fr).

OBJECTIF

Au moins une source d'enrichissement licite intégrée, créant une valeur difficilement reproductible.

TÂCHES

- Identifier une/des sources d'enrichissement licites (e-mail pro, dirigeant, SIRET).
- Intégrer l'enrichissement à la fiche et au CRM.
- Réactiver une action e-mail réelle une fois la donnée disponible (cf. fonction retirée).
- Évaluer l'aspect licence/conformité de chaque source.

CRITÈRES D'ACCEPTATION

- ✓ Au moins une source d'enrichissement licite est intégrée et exploitable.
- ✓ Une action e-mail réelle est disponible quand la donnée existe.

DÉPENDANCES

Stratégique ; conforme RGPD (M13.4).

FICHIERS IMPACTÉS : backend (service enrichissement) · frontend/app/hooks/useProspects.ts · DetailClient.tsx

M13.2 Fournisseur Places de repli + alerting actif

Majeur

CONSTAT

Résidu de PW-29 : repli géocodeur Google → Nominatim et circuit-breaker en place, mais **aucun fournisseur de repli pour Places lui-même**, et l'alerte reste en logs. Une suspension de la clé Google fait taire le produit.

OBJECTIF

Une panne Places dégrade proprement (alerte active + repli si possible), sans silence.

TÂCHES

- Abstraire la source de places pour permettre un fournisseur de repli.
- Brancher l'alerte active (M10.4) sur les pannes Places.
- Documenter la procédure de bascule (runbook M12.3).

CRITÈRES D'ACCEPTATION

- ✓ Une panne Places déclenche une alerte active.
- ✓ Un mécanisme de repli/dégradation explicite existe.

DÉPENDANCES

S'appuie sur M10.4 et M12.3.

FICHIERS IMPACTÉS : backend/src/services/places.ts · backend/src/services/geocode.ts

M13.3 API publique & intégrations CRM externes

Mineur

CONSTAT

Aucune intégration verrouillante (export CSV seulement). Les intégrations CRM externes et une API publique augmentent la valeur et la rétention.

OBJECTIF

Le produit s'intègre à l'écosystème du client (export enrichi, API, connecteurs CRM).

TÂCHES

- Définir une API publique authentifiée (clé/scopes).
- Prioriser un/deux connecteurs CRM cibles.
- Documenter l'API.

CRITÈRES D'ACCEPTATION

- ✓ Une API publique authentifiée est disponible et documentée.
- ✓ Au moins un connecteur CRM fonctionne de bout en bout.

DÉPENDANCES

Phase 2.

FICHIERS IMPACTÉS : backend (API publique) · docs

M13.4 Anti-rebinding, opt-out tiers RGPD & avis juridique export

Mineur

CONSTAT

Angles morts signalés par l'audit : fenêtre théorique de DNS-rebinding (TOCTOU entre `lookup()` et `fetch()` dans `websiteChecker`), absence d'opt-out pour les personnes prospectées (distinct de la suppression du compte utilisateur), base légale RGPD à documenter, et arbitrage juridique sur l'export de nom/adresse au regard des conditions Google Places. Ces points relèvent en partie d'un avis juridique, pas du code.

OBJECTIF

Les risques résiduels de sécurité et juridiques sont traités ou explicitement couverts par un avis.

TÂCHES

- Épingler l'IP résolue lors du fetch de vérification (anti-rebinding) ou passer par un proxy de sortie cloisonné.
- Mettre en place un opt-out / suppression sur demande pour les personnes prospectées.
- Documenter la base légale RGPD et la durée de conservation.
- Obtenir un avis juridique sur l'export (Google Places ToS, RGPD).

CRITÈRES D'ACCEPTATION

- ✓ Le fetch de vérification n'est plus vulnérable au rebinding (IP épinglée).
- ✓ Un opt-out tiers existe ; la base légale et la conservation sont documentées.
- ✓ Un avis juridique encadre l'export.

DÉPENDANCES

Partiellement hors code (juridique) — angle mort assumé.

FICHIERS IMPACTÉS : `backend/src/services/websiteChecker.ts` · `frontend/app/privacy/page.tsx`
· docs (juridique)

Checklist post-déploiement

À cocher avant chaque ouverture (bêta puis public). Couvre les barrières plateforme hors dépôt (angles morts assumés) autant que les correctifs applicatifs.

- ☐ Secrets renseignés (Vercel + Railway) : Clerk, Stripe (clés + webhook secret), Resend, Google, Redis, Postgres, Sentry, Mixpanel
- ☐ Quotas QPD par SKU configurés dans la console Google Cloud + alertes budgétaires actives
- ☐ Clé API Google restreinte (référents/IP serveur, APIs autorisées)
- ☐ Webhooks Stripe configurés et testés (signature, idempotence)
- ☐ Clerk : domaines, redirections, politique de mot de passe, anti-bot, OAuth Google
- ☐ CSP en production sans `unsafe-inline/unsafe-eval`, domaines Clerk/Stripe autorisés
- ☐ `/api/usage` global protégé ou retiré ; `PATCH /plans/me` verrouillé derrière abonnement
- ☐ Quotas Google/utilisateur et rate-limiter autoritatifs (Redis/DB), partagés multi-instance
- ☐ Circuit-breaker en état partagé ; alerte active branchée (panne Google, seuil quota, paiement échoué)
- ☐ Resend : domaine d'envoi vérifié ; e-mails transactionnels testés
- ☐ Sentry actif (back+front) ; Mixpanel et Vercel Analytics collectent
- ☐ Sauvegardes Postgres et politique de purge (30 j données Google, inactifs > 12 mois) actives
- ☐ Politique de confidentialité publiée ; opt-out tiers disponible ; avis juridique export obtenu
- ☐ Suite de tests verte en CI, incluant auth Clerk, webhook Stripe et test de charge

Glossaire

Acronymes et termes employés dans la feuille de route.

Bêta fermée	Ouverture à un cercle restreint d'utilisateurs de confiance, avant lancement public.
Circuit-breaker	Disjoncteur logiciel qui coupe temporairement les appels à un service en panne pour éviter de l'aggraver.
Clerk	Fournisseur d'authentification managé (sessions, OAuth, MFA, reset, anti-bot).
CSP	Content-Security-Policy : en-tête HTTP limitant les sources de scripts/styles, défense anti-XSS.
DAG	Graphe orienté acyclique : ici, le graphe de dépendances entre milestones.
DNS-rebinding	Attaque exploitant un changement de résolution DNS entre la validation et l'usage d'une adresse (TOCTOU).
FieldMask	Liste des champs demandés à l'API Google Places ; influe sur le palier tarifaire.
httpOnly	Attribut d'un cookie le rendant illisible par JavaScript (protège contre le vol par XSS).
Idempotence	Propriété d'un traitement qui, rejoué, ne produit pas d'effet en double (webhooks).
Métering	Mesure de la consommation par utilisateur, base de la facturation à l'usage.
MFA	Authentification multi-facteur (ex. code TOTP en plus du mot de passe).
Place ID	Identifiant durable d'un lieu Google ; seul champ généralement conservable durablement.
QPD	Queries Per Day : plafond de requêtes quotidiennes côté Google Cloud.
Rate-limiter	Composant limitant le nombre de requêtes par fenêtre de temps.
reCAPTCHA v3	Anti-bot Google silencieux basé sur un score, sans défi visible.
Resend	Service d'envoi d'e-mails transactionnels via API.
SSRF	Server-Side Request Forgery : faire émettre au serveur des requêtes vers des ressources internes.
Turnstile	Anti-bot Cloudflare, mécanisme natif de Clerk (alternative à reCAPTCHA).
Webhook	Appel HTTP entrant émis par un service tiers (ex. Stripe) pour notifier un événement.

Les constats reposent sur la lecture effective du dépôt [sofianehms/prospectweb](#) au commit [c7fb6d1](#) (« Nosite v0.1 ») et sur les trois audits fournis (22, 23 et 24 juin 2026, portant sur [c94cef9](#)). Les références [fichier:ligne](#) ont été recoupées avec l'état actuel du code. Les criticités et recommandations relèvent de l'analyse ; la configuration d'infrastructure, la tarification Google et la conformité juridique finale restent des angles morts signalés.